Why Agent Plugins Are Not a Security Boundary: Inverting Control from In-Process Hooks to a Kernel-Enforced Agent Runtime

Hui Zhao
Heurion Research
Rotterdam, Netherlands
huizhao@heurion.org

Preprint — November 2026

Abstract

Autonomous CLI coding agents—Claude Code (`claude`), OpenCode (`opencode`), Google Antigravity (`agy`), and Nous Hermes (`hermes`)—rely on in-process governance plugins (`PreToolUse` hooks, additive MCP servers, and prompt rules). We show that treating an in-process plugin as a security boundary violates Anderson's Reference Monitor guarantees of complete mediation, tamper-resistance, and verifiability whenever the agent runs under the developer's UNIX UID. Auditing four production CLI agents reveals six structural bypass classes (`V1`–`V6`): workspace escape and unapproved host writes (`V1`); unconfined socket egress (`V2`); host credential, Keychain, and environment key theft (`V3`); hook/config self-tampering (`V4`); repository code execution during read-only `git status/diff` via `core.fsmonitor` (`V5`); and same-UID self-approval or post-diff TOCTOU swaps (`V6`).

We present **Aegis Enforced Runtime** (`aegis-os run -- <agent>`), which inverts control so the OS supervisor confines the unmodified agent inside Linux `bubblewrap` (`--bind-fd`) or macOS Seatbelt before `execve()`. Aegis combines Copy-on-Write shadows with a read-only `.git` view and `.aegis-bin/git` shim, read-only config over-mounts, a split-plane L4/L7 egress proxy that injects API keys at the host boundary without TLS MITM, dual-token `SessionBroker` isolation, and frozen-stage SHA-256 commit pinning. Across 30 deterministic payloads on macOS and Linux plus 54 executed live runs (60-cell matrix) of all four agents, Aegis reduces bypass coverage from 73.3%–93.3% (and 80.0% in `nerdctl` devcontainers) to **0.0%** and live repository-trap compromises from 100% to **0%**, while saving 99.05% of APFS disk allocation on 500 MB packed Git repos, adding +0.221 ms loopback TTFT overhead, and preserving 100% multi-file coding pass rates.

1 Introduction

Command-line autonomous coding agents—including Anthropic Claude Code (`claude`), OpenCode (`opencode`), Google Antigravity (`agy`), Nous Hermes (`hermes`), OpenAI Codex CLI [16], SWE-agent [28], and OpenHands [23]—have become core developer infrastructure [11, 3]. Unlike passive code completion tools, these agents execute as autonomous user-space processes that edit source trees, run build scripts and test suites, inspect Git state, and communicate with external APIs.

To govern agent actions without constant manual prompting, both vendors and security tools have converged on an *in-process plugin paradigm*: registering an MCP server [3], configuring a `PreToolUse` or `tool.execute.before` hook in user dotfiles (`~/.claude/settings.json`, `~/.gemini/config/hooks.json`, or `~/.config/opencode/plugins/`), and placing natural-language routing instructions in `CLAUDE.md` or `AGENTS.md`.

In our companion work on Aegis OS [31], moving MCP-mediated file edits and shell commands into a transactional Copy-on-Write (CoW) shadow workspace (`aos-vfs`) eliminated 90.7%–99.88% of human confirmation prompts across 28,087 tool calls. Crucially, however, [31] explicitly assumed that client-side hooks reliably route agent mutations into the runtime.

1.1 Why In-Process Plugins and RW Sandboxes Fail as Reference Monitors

We ask a foundational systems security question: *Can an in-process agent plugin, hook script, or MCP server ever serve as a true security boundary when the agent CLI itself runs as an unconfined host process?*

By Anderson's classic Reference Monitor formula-

tion [2] and Saltzer and Schroeder's protection principles [21], a valid enforcement mechanism must satisfy three properties:

1. $\mathcal{M}_1$ **(Complete Mediation):** Every security-relevant access to host files, credentials, IPC endpoints, and network sockets must traverse the monitor.
2. $\mathcal{M}_2$ **(Tamper-Resistance):** The untrusted subject (the LLM-driven agent process tree) must not be able to modify, disable, or forge the monitor's configuration, hooks, or approval state.
3. $\mathcal{M}_3$ **(Verifiability):** The enforcement boundary must be small, deterministic, and auditable independently of the rapidly evolving application code of third-party agent CLIs.

Auditing four production CLI agents under the developer's UNIX UID (Table 1) shows that in-process governance violates all three axioms, yielding six structural bypass classes (`V1`–`V6`, formalized in §2):

- **Failure of Complete Mediation ($\neg\mathcal{M}_1 \Rightarrow$ `V1`, `V2`, `V3`):** MCP is an *additive* tool protocol, not a mandatory syscall interposer. Built-in native file tools, repository-configured `.mcp.json` servers, warn-only plugin hooks, and innocuous project scripts (`python3 scripts/verify_env.py`) execute arbitrary VFS, socket, and IPC syscalls invisible to command-string regexes—enabling unapproved host writes (**V1**), direct socket exfiltration (**V2**), and host credential, Keychain, or `environ` API-key theft (**V3**).
- **Failure of Tamper-Resistance ($\neg\mathcal{M}_2 \Rightarrow$ `V4`, `V6`):** Because the agent inherits ambient UID write permissions across `$HOME` and the project root, unguarded writes or child scripts can overwrite hook/-plugin configs to disarm governance (**V4**), or forge same-UID loopback approvals and perform post-diff TOCTOU swaps (**V6**).
- **Failure of Verifiability in "Read-Only" Tooling ($\neg\mathcal{M}_3 \Rightarrow$ `V5`):** Usable hook policies allowlist routine inspection commands (`git status`, `git diff`, `pytest`), yet Git evaluates repository-local `.git/config` (`core.fsmonitor`, `diff.external`)—turning "read-only" inspection into host code execution (**V5**).

Crucially, even existing vendor subprocess sandboxes (`claude sandbox.enabled` [4], `agy --sandbox=seatbelt`) and standard developer containers (`nerdctl run -v $(pwd):/workspace`) fail to close these vectors: subprocess sandboxes wrap only child shell invocations while leaving parent tools and `$HOME` exposed, and both approaches mount the live host repository (including `.git` and `.mcp.json`) *read-write* while passing raw LLM API keys into `environ`.

1.2 Inverting Control to a Parent-Process Kernel Runtime

Our central thesis is architectural: *a security runtime must not reside inside an agent as an optional plugin; the agent must execute inside the runtime as an unprivileged child process tree.* Instead of trusting the agent binary's internal hook dispatcher, the operator invokes:

```
aegis-os run -- <agent> [args...]
```

Before `execve()` enters the agent binary, the parent `aegis-os` supervisor establishes kernel-enforced containment (Linux `bubblewrap` [18] user/mount/PID/IPC/network namespaces with `--bind-fd`, or macOS XNU Seatbelt MAC). Making parent-process containment practical for real interactive coding agents requires resolving four systems tensions: (1) preserving canonical workspace paths without copying multi-hundred-megabyte `.git/objects/pack` stores or exposing mutable `.git/config` hooks; (2) allowing session OAuth/settings updates via atomic `rename(2)` while kernel-locking hook/plugin configs and blocking macOS Keychain/LaunchServices IPC escapes; (3) scrubbing plaintext API keys from `environ` and inspecting outbound model payloads without TLS Man-in-the-Middle (MITM) certificate breakage; and (4) enforcing human commit approval without corrupting Alternate Screen TUIs or permitting post-diff Time-of-Check to Time-of-Use (TOCTOU) swaps.

1.3 Contributions

This paper makes five contributions:

- **Reference-Monitor Audit and Six-Vector Bypass Taxonomy (§2):** We audit four production CLI coding agents (`claude v2.1.319`, `opencode v2.2.6`, `agy v1.5.3`, and `hermes v0.23.0`), formalizing six structural bypass classes (`V1`–`V6`) across plugin hooks, vendor subprocess sandboxes, and bind-mounted developer containers.
- **Kernel Filesystem, Config, and Git Containment (§3):** We design Aegis's filesystem containment plane (`crates/aos-cli/src/launcher.rs`), combining `fcntl(F_DUPFD, 3)` inherited descriptor mounts (`--bind-fd`), macOS Keychain/LaunchServices Mach confinement, layered agent homes with read-only hook/plugin locks, and a split read-only `.git` view paired with a mode-`0555` `.aegis-bin/git` shim (`V1, V3, V4, V5`).
- **Split-Plane L4/L7 Egress Gateway and Boundary Secret Vault (§4):** We pair an L4 `CONNECT` domain allowlist with an L7 `Base URL` reverse proxy (`crates/aos-egress/src/lib.rs`) that substitutes `$AOS_SECRET_*` placeholders at the host hop and runs 64 KiB outbound DLP without TLS MITM (`V2, V3`).

Table 1: **Architectural Audit of Four Installed Production CLI Coding Agents** across Bypass Classes V1–V6 (§2; verified via CLI inspection, local configuration schemas, and source/log auditing on our evaluation host) versus **Aegis Enforced Runtime (aegis-os run)**.

Agent (Version)	Hook & Native Sandbox	V1/V2: FS & Egress Mediation	V3: Secrets & IPC Surface	V4/V5: Config & .git RCE	V6: Approval & Commit Gate
Claude Code (2.1.319)	settings.json PreToolUse; sandbox.enabled [4]	Hooks regex-bypassed via repo scripts; native sandbox mounts host repo RW	Keychain / OAuth token; raw env vars exposed to child processes	settings.local.json & .mcp.json editable; core.fsmonitor runs in repo	Direct in-place host edits without frozen SHA-256 diff commit gate
OpenCode (v2.2.6)	JS plugin tool.execute.before; *no OS sandbox*	No built-in OS sandbox; serve/attach delegates to host daemon	Plaintext DEEPSEEK_API_KEY in environ & auth.json	.config/opencode/plugins/*.js & opencode.json writable; fsmonitor runs	No out-of-sandbox commit gate; same-UID loopback HTTP open
Antigravity (1.5.3)	hooks.json protojson (exit 2); --sandbox=seatbelt	Strict shell hook breaks test runs; vendor Seatbelt allows sibling & repo RW	OAuth file + SecurityAgent Keychain IPC (/usr/bin/security)	Vendor Seatbelt permits hooks.json & repo config writes; fsmonitor open	User-space router guard bypassed if native shell executes directly
Nous Hermes (v0.23.0)	security-guidance (warn-only); *no OS sandbox*	Bundled plugin warns once without blocking; HERMES_SAFE_MODE regex-only	Plaintext keys in ~/.hermes/.env; login shell resets $HOME via pw_dir	~/.hermes/config.yaml writable; .git/config fsmonitor runs	No kernel sandbox or principal-separated approval broker
Aegis Enforced (aos run)	**Parent bwrap --bind-fd / XNU Seatbelt**	**CoW shadow remap; netns/port-pinned L4+L7 egress**	**Host creds masked; Mach Keychain & launchd denied; $AOS_SECRET_***	**--ro-bind-fd / Seatbelt config locks; 0555 .aegis-bin/git shim**	**agent.token ≠ human.token; frozen-stage SHA-256 commit**

- **Principal-Separated Broker, Frozen-Stage Commit Gate, and Invariants (§5):** We pair a constant-time dual-token SessionBroker (agent.token ≠ human.token) with TUI-safe out-of-band escalations, frozen-stage SHA-256 commit pinning, and six DomainProfile verifiers (V6), proving five security invariants ($\mathcal{I}_1$–$\mathcal{I}_5$; Invariants 1–5).
- **Two-Layer Cross-Platform & Live-Agent Evaluation (§6):** Across 30 deterministic payloads on macOS Seatbelt and Ubuntu 24.04 bwrap/nerdctl, 4 feature-gated Rust ablations (AOS_ABLATE), 3 filesystem CoW benchmarks (APFS, ext4, btrfs), and 54 executed live cloud-API runs (60-cell matrix) across all four production CLI agents, Aegis achieves **0.0%** bypass coverage and **0%** live compromise while preserving **8/8 (100%)** benign coding pass rates.

2 Threat Model and Six Structural Bypass Classes

2.1 System Model, Adversary, and TCB

We consider a workstation or build server where an operator runs a CLI coding agent $A \in \{\texttt{claude}, \texttt{opencode}, \texttt{agy}, \texttt{hermes}\}$ on a local Git repository $\mathcal{W}_{\text{host}}$ under UNIX account u.

Adversary Capabilities. Following standard indirect prompt-injection and malicious-repository threat models [10, 8, 30], the repository $\mathcal{W}_{\text{host}}$ or external context (issues, dependencies, build logs, test fixtures) may contain adversarial instructions, trojanized helper scripts (e.g., scripts/verify_env.py), or malicious repository metadata (.git/config, .mcp.json). Unlike [31], we grant the subverted agent tree full access to *every syscall, native tool, subshell, IPC channel, and socket permitted by the OS kernel to A and its descendants.*

Trusted Computing Base (TCB). We trust the host OS kernel (Linux $\geq$ 6.1 with user/mount/network namespaces and bubblewrap [18], or macOS XNU Seatbelt) and the parent aegis-os binary. We do *not* trust the agent binary A, its in-process hook/plugin runner, repository configs, or child build/test scripts (build.rs, pytest). Denial-of-service, kernel privilege-escalation zero-days, and semantic prose summarization over authorized LLM inference channels (§7) are out of scope.

2.2 The Six Bypass Classes (V1–V6)

Auditing the exact installed versions of our four target agents (Table 1 and Figure 1a) yields six concrete bypass classes:

V1: Workspace Escape and Unapproved Host-Tree Writes. Command-string hooks (PreToolUse on Bash/run_command) inspect surface shell syntax rather than resolved VFS paths. An agent invoking an uncovered native file tool, spawning a repository-configured .mcp.json child server, or running an innocuous test helper (python3 scripts/verify_env.py) writes directly to sibling host directories (../sibling)

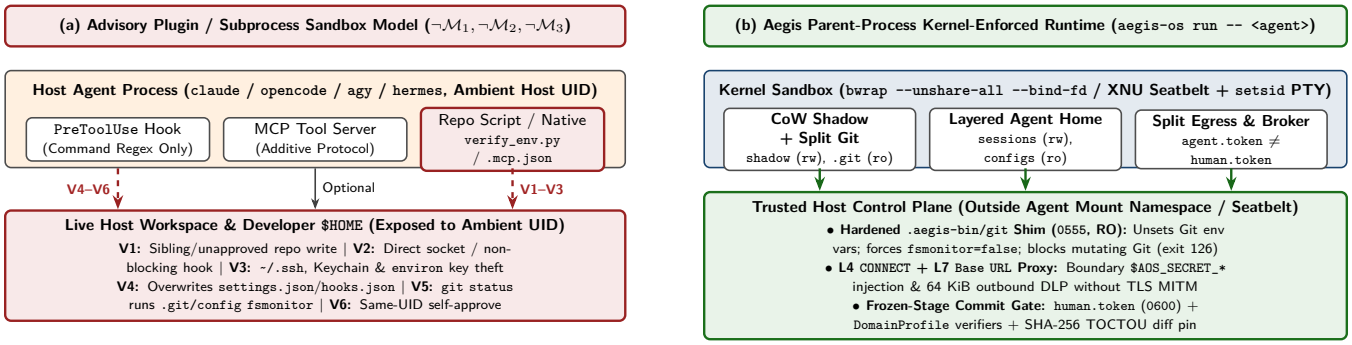


Figure 1: **Architectural Inversion of Control.** (a) Under in-process hooks or RW-workspace subprocess sandboxes, repository helper scripts, non-blocking plugin hooks, plaintext environment keys, writable config files, and `.git/config fsmonitor` hooks bypass governance (V1–V6). (b) **Aegis Enforced Runtime** confines the entire agent process tree inside a parent kernel sandbox *before* `execve()`, isolating writes in a CoW shadow, mounting `.git` and agent configs read-only, injecting API keys at the host L7 proxy boundary, and requiring `human.token` plus frozen-stage SHA-256 verification before host promotion.

or mutates $\mathcal{W}_{\text{host}}$ in place without transactional review. Crucially, vendor subprocess sandboxes (`claude sandbox.enabled` [4], `agy --sandbox=seatbelt`) and bind-mounted developer containers (`nerdctl run -v $(pwd):/workspace`) mount $\mathcal{W}_{\text{host}}$ *read-write*, permitting immediate unapproved host-repository modification and `.mcp.json` injection.

V2: Unconfined Network Egress and Non-Blocking Hook Paths. Agents without built-in network sandboxes (`opencode`, `hermes`, and standard `nerdctl` containers with default bridge networking) allow arbitrary exfiltration via `curl --noproxy '*'`, Python `urllib`, raw TCP sockets, or background subshells. Moreover, auditing `hermes v0.23.0` revealed that its bundled `security-guidance` plugin records a session key and *warns once without blocking execution*, while its `plugin-guard-v12` runs in-process with explicit tool exemptions; even with `HERMES_SAFE_MODE=1` or `claude`/`opencode` regex hooks enabled, indirect execution through a project script bypasses string-matching filters completely.

V3: Host Credential, Keychain, and Sandbox API-Key Theft. Under ambient UID u, child tools read `~/.ssh/id_ed25519`, `~/.aws/credentials`, and agent OAuth files (`antigravity-oauth-token`, `~/.hermes/.env`), or dump plaintext `ANTHROPIC_API_KEY` / `DEEPSEEK_API_KEY` from `os.environ` (which vendor sandboxes and devcontainers must pass into the process so the agent can call its LLM). On macOS, child processes can also invoke `/usr/bin/security`, talk to Keychain `SecurityAgent` or LaunchServices (`open -a`, `osascript`, `launchctl`) over Mach IPC to spawn unsandboxed children under `launchd`, or attach to an unconfined host daemon (`opencode serve`/`attach`).

V4: Self-Modifying Hook and Plugin Configuration. Hook definitions reside in user-writable files: `~/.claude/settings.json`, `.claude/settings.local.json`, `~/.gemini/config/hooks.json`, `~/.gemini/antigravity-cli/settings.json`, `~/.config/opencode/plugins/*.js`, and `~/.hermes/config.yaml`. Any unhooked write or project script can truncate `settings.json`, drop a rogue `opencode` plugin, or rewrite `config.yaml`, permanently disarming in-process governance.

V5: Repository-Triggered RCE in "Read-Only" Git Commands. Developer workflows require `git status`, `git diff`, and `git log`. However, Git automatically parses `.git/config` inside the repository; setting `core.fsmonitor` or `diff.external` triggers arbitrary command execution whenever the agent or user runs `git status`. Likewise, writable `.git/hooks/post-checkout`, `git -c core.fsmonitor=...`, or `GIT_CONFIG_PARAMETERS` achieve code execution through ostensibly read-only Git invocations.

V6: Same-UID Control-Plane Self-Approval and Post-Diff TOCTOU. If a local approval daemon authenticates callers only by UNIX UID or a single session token, an in-sandbox process can approve its own escalation over loopback HTTP/Unix sockets, invoke the approval CLI directly, swap shadow files in the $[t_{\text{diff}}, t_{\text{commit}}]$ window, spoof terminal diff output via ANSI CSI (`\x1b[2K`) or Unicode Bidi (`U+202E`) escapes, or replay a consumed lease.

3 Kernel Filesystem, Config, and Git Containment

To close V1–V6 without modifying third-party agent binaries, **Aegis Enforced Runtime** acts as the parent OS supervisor prior to `execve()` (Figure 1b), partitioning mediation across three orthogonal enforcement planes:

1. **Filesystem, Config, and Git Containment (§3):** Remaps workspace writes into a CoW shadow (**V1**), masks host credentials and Keychain/TTY IPC (**V3**), kernel-locks hook and plugin configurations (**V4**), and enforces a split read-only `.git` view (**V5**), establishing Invariants $\mathcal{I}_1$ and $\mathcal{I}_2$.

2. **Split-Plane Egress and Boundary Secret Vault (§4):** Confines outbound sockets to a two-plane host proxy (**V2**) and replaces static LLM API keys in sandbox `environ` with boundary-injected placeholders (**V3**), establishing Invariant $\mathcal{I}_3$.

3. **Principal-Separated Broker and Frozen-Stage Commit Gate (§5):** Separates `agent.token` from `human.token` for out-of-band TUI-safe approvals and pins promoted diffs via a frozen SHA-256 staging protocol (**V6**), establishing Invariants $\mathcal{I}_4$ and $\mathcal{I}_5$.

`aegis-os run` defaults to `--enforcement enforced` (with `advisory` labeled `NOT a security boundary` and `isolated` backed by `MicroVmSandboxDriver` [1]). Before spawning any child process, `validate_workspace_roots()` rejects `/`, any ancestor of `$HOME`, or overlapping runtime paths, and `probe_bwrap_userns()` fails closed if kernel namespace setup is restricted (such as missing AppArmor `userns` profiles on Ubuntu 24.04+; see §7).

3.1 Default-Deny Mount Namespace, `--bind-fd` Remapping, and Mach Confinement

Linux `--ro-bind / /` + Canonicalized `--tmpfs` + `--bind-fd`. To support polyglot CLIs (Node.js, Bun, Go, Rust, Python) across Ubuntu, Fedora Silverblue, and NixOS without exposing user data, `LauncherPlan::linux_args()` starts with `--die-with-parent --unshare-all --ro-bind / / --proc /proc --dev /dev --new-session`, masks `/etc/shadow`, `/etc/sudoers*`, and `/etc/ssl/private`, and mounts fresh `--tmpfs` instances over canonicalized, depth-sorted user roots (`/tmp, /home, /root, /run, /var, /mnt, /media`). Because path-based bind sources inside `/tmp` or `/home` disappear once `--tmpfs` is mounted, `fd_mount()` opens host directories in the parent process, duplicates descriptors via `fcntl(raw, F_DUPFD, 3)`, and passes `--bind-fd <fd> <project>` to `bwrap`, mounting the CoW shadow ($\mathcal{W}_{\text{shadow}}$) directly at the *exact canonical project path* while leaving $\mathcal{W}_{\text{host}}$ untouched (**V1**).

macOS XNU Seatbelt + Mach IPC / Keychain Confinement. On macOS (which lacks unprivileged `CLONE_NEWNS`), `generate_macos_seatbelt_profile()` enforces `(deny file-write*)`, permitting writes only inside $\mathcal{W}_{\text{shadow}}$, the isolated agent home $\mathcal{H}_{\text{agent}}$, and `/tmp`, and denies `file-read*` across 20 host credential patterns ($\mathcal{S}_{\text{host}}$: `.ssh`, `.aws`, `.gnupg`, `.kube`, `.config/gh`, `.config/gcloud`, `Library/Keychains`, `docker.sock`, `.lima`). To close macOS-specific IPC escape vectors (**V3**), the profile enforces:

```
(deny appleevent-send)
(deny process-exec
  (literal "/usr/bin/security") (literal "/bin/launchctl")
  (literal "/usr/bin/open")     (literal "/usr/bin/osascript"))
(deny mach-lookup
  (global-name "com.apple.security.agent")
  (global-name "com.apple.security.agent.login")
  (global-name-regex #"^com\.apple\.(coreservices\.|lsd\.)"))
```

Live testing of `agy v1.5.3` underscored why Mach-level Keychain confinement is essential: `agy`'s Go keyring library invokes `/usr/bin/security add-generic-password -U -s antigravity` on OAuth refresh; when `$HOME` points to $\mathcal{H}_{\text{agent}}$ without a login keychain, macOS `Security.framework` contacts `com.apple.security.agent` and spawns modal GUI prompts (*"Keychain Not Found"*) outside the terminal. Blocking `/usr/bin/security` (paired with a `0555 .aegis-bin/security` exit-44 stub) and denying `com.apple.security.agent*` eliminates both host Keychain exfiltration and GUI prompt stalls while preserving `com.apple.trustd` TLS validation. Finally, `run_pty_command()` allocates a dedicated raw-mode PTY session via `setsid()` and strips `opencode --server/attach`, preventing daemon and controlling-TTY escapes.

3.2 Layered Agent Home, Read-Only Config Locks, and CLI Compatibility

`seed_agent_home()` constructs $\mathcal{H}_{\text{agent}}$ in three layers to neutralize **V4** without breaking interactive agent state: (1) symlink-safe selective copying of small metadata files (`SEEDED_AGENT_FILES`, skipping multi-hundred-megabyte session DBs, `venvs`, `node_modules`, and live daemon lockfiles); (2) writable session subdirectories; and (3) kernel read-only locks (`--ro-bind-fd` on Linux; Seatbelt `(deny file-write*)` rules on macOS, backed by user-space router guards in `routing.rs`) over executable governance configs $\mathcal{C}_{\text{agent}}$: `.claude/settings*.json`, `.gemini/config/{hooks,mcp_config}.json`, `.hermes/config.yaml` (unlocked only for explicit

hermes config/login setup), plugin/hook directories (.claude/hooks, .gemini/hooks, .hermes/hooks, .config/opencode/plugins), and project-level .mcp.json and opencode.json*. Where an agent requires atomic rename(2) on session-preference files during interactive OAuth login (e.g., .claude.json and agy's settings.json, which fail with EBUSY if single-file bind-mounted), mutations remain confined inside $\mathcal{H}_{\text{agent}}$ and cross to the host only through a narrow *OAuth declassification gate* (sync_credentials()): the parent supervisor opens only the active binary's store with O_NOFOLLOW ($\leq$ 64 KiB, rejecting symlink components so a sandboxed child cannot trick the parent into copying host files), re-serializes validated JSON ASTs to fresh 0600 tempfiles, and merges strictly allowlisted OAuth/onboarding fields while dropping hooks, mcpServers, security, coreTools, and unknown keys (disableable via AOS_NO_CREDENTIAL_SYNC=1).

Running all four production agents live surfaced three OS-level compatibility constraints that we resolved in launcher.rs: (i) *AF_UNIX path-length caps* (104/108-byte sun_path overflow under deep CoW directories, solved via lifecycle-bound /tmp/aos-{ctl,sbx,xr}-<12hex> sockets); (ii) *single-port loopback pinning* on macOS Seatbelt (pre-allocating one random-authenticated loopback port per session for opencode/hermes internal HTTP servers rather than opening wildcard loopback access); and (iii) *Hermes login-shell isolation* (exporting HERMES_HOME + HERMES_DISABLE_LAZY_INSTALLS=1 and borrowing hermes-agent/runtime/tools read-only so bash -lc neither resets $HOME via pw_dir nor writes .prepare.lock to the host).

3.3 Split Read-Only .git View and Hardened .aegis-bin/git Shim

Copying .git/objects/pack on every session wastes hundreds of megabytes, whereas mounting .git read-write exposes .git/config and .git/hooks, and mounting .git read-only breaks git status (index.lock). Aegis resolves this three-way tension (**V5**):

1. **Read-Only .git + Private GIT_INDEX_FILE:** Host <project>/.git ($\mathcal{G}_{\text{host}}$) is mounted read-only (--ro-bind-fd on Linux; GIT_DIR=<project>/.git + GIT_WORK_TREE=<shadow> under Seatbelt write-deny on macOS), while only the compact .git/index is copied to .aegis-git-index ($\mathcal{G}_{\text{idx}}$) via GIT_INDEX_FILE with GIT_OPTIONAL_LOCKS=0.

2. **Read-Only 0555 .aegis-bin/git Wrapper:** Prepended to PATH inside a mode-0555 read-only directory, our wrapper unsets GIT_CONFIG_PARAMETERS, GIT_CONFIG_COUNT, GIT_EXTERNAL_DIFF, GIT_SSH_COMMAND, and GIT_EXEC_PATH; rejects caller-supplied -c, --config-env, and --exec-path flags anywhere in argv; injects -c core.fsmonitor=false -c core.hooksPath=/dev/null -c protocol.file.allow=never (plus --no-ext-diff --no-textconv on diff/log/show); and aborts mutating Git subcommands (commit, push, reset, config) with exit code 126.

4 Split-Plane Egress Gateway and Boundary Secret Vault

4.1 L4 CONNECT Allowlist and L7 Base URL Reverse Proxy

TLS-terminating MITM proxies break certificate-pinned Go/Rust/Bun agent binaries and stall Server-Sent Events (SSE). To mediate network sockets (**V2**) and API credentials (**V3**) without TLS MITM, start_egress() implements a two-plane host gateway (Figure 2):

- **Plane 1 (L4 CONNECT Domain Allowlist):** Linux CLONE_NEWNET removes external interfaces, relaying traffic through egress-shim over a token-authenticated Unix socket to AegisEgressGateway; macOS Seatbelt restricts network-outbound to the proxy port. Allowlisted HTTPS domains (github.com, crates.io, pypi.org, npmjs.org) use transparent TCP splicing without TLS termination.

- **Plane 2 (L7 Base URL Reverse Proxy & Outbound DLP):** launcher.rs rewrites model SDK base URLs (ANTHROPIC_BASE_URL, OPENAI_BASE_URL, GOOGLE_GEMINI_BASE_URL, plus opencode and hermes provider configs) to http://<proxy>/__aegis/base/<provider>. At the host hop, crates/aos-egress/src/lib.rs normalizes RFC 3986 paths against capability scopes, scans query strings and up to 64 KiB of request bodies via inspect_payload_dlp() for private keys, AWS/GitHub tokens, raw secrets, or reflected $AOS_SECRET_* placeholders, injects the real API key, and streams upstream TLS SSE chunks immediately.

Adversarial testing of AegisEgressGateway uncovered and fixed three L7 proxy bugs prior to live deployment: (1) RFC 9112 absolute-URI vs. Host-header authority mismatch (domain-fronting); (2) non-Gemini query-string $AOS_SECRET_* substitution before query DLP; and (3) release-build exposure of a test-only X-Aegis-Upstream routing header (now gated behind #[cfg(test)]).

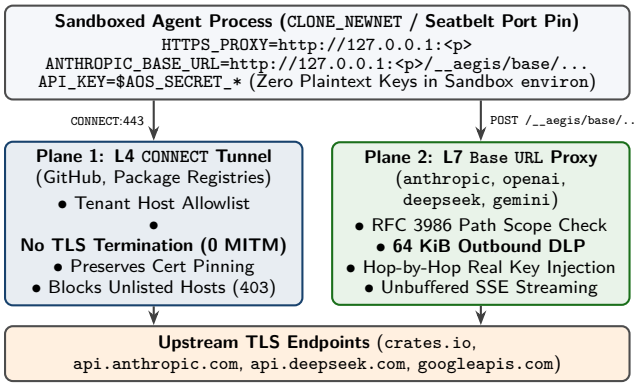


Figure 2: **Split-Plane Egress Gateway (`AegisEgressGateway`).** Plane 1 tunnels allowlisted HTTPS traffic without TLS MITM; Plane 2 proxies model APIs over loopback HTTP, scanning 64 KiB bodies for secret exfiltration and injecting real API keys only at the upstream TLS hop.

4.2 Boundary Secret Vault vs. Scoped Session OAuth

Static long-lived API keys (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `DEEPSEEK_API_KEY`, plus provider entries in `~/.hermes/.env` and `opencode`'s `auth.json`) are harvested on the host prior to `execve()` and replaced in sandbox `environ` and config files with scoped `$AOS_SECRET_*` placeholders (wired to `aos_core::ablated("secret_vault")` only in test-ablation builds). By contrast, interactive OAuth clients (`claude /login`, `agy`, `codex`, and `hermes` Portal OAuth) rotate refresh/access token pairs in local credential files and fail if overridden by static environment tokens. Aegis therefore unlinks all foreign-agent OAuth stores (including `.gemini` and `antigravity-cli` token files) from $\mathcal{H}_{\text{agent}}$ during seeding and restores only the active client's OAuth store (`0600`) inside its isolated $\mathcal{H}_{\text{agent}}$, routing inference requests through Plane 2 L7 DLP and gating host write-back through active-client, `O_NOFOLLOW` schema-allowlisted `sync_credentials()` (§3).

5 Principal-Separated Broker, Frozen-Stage Commit Gate, and Security Invariants

5.1 Dual-Token Session Broker and Out-of-Band Escalation

To close control-plane self-approval and TUI prompt-spoofing vectors (**V6**), `SessionBroker::start()` (`crates/aos-cli/src/session_broker.rs`) mints two independent 256-bit CSPRNG tokens:

1. **`agent.token`:** Exported via `AOS_SESSION_TOKEN` and accepted via constant-time `token_eq` *only* on adapter/MCP routes (`/v1/session`, `/v1/adapter/*`, `/v1/mcp`).
2. **`human.token`:** Stored in `session_dir/human.token` (`0600`) on the host outside the sandbox mount namespace and denied by Seatbelt `tenant_rules`. Human routes (`/v1/escalations`, `/v1/escalations/:id/decide`, `/v1/leases`, `/v1/audit`) reject `agent.token` with `HTTP 401`, while `OneShot` escalation tickets cannot be replayed once consumed.

Because full-screen CLI agents activate the VT100 Alternate Screen (`\x1b[?1049h`) and raw TTY mode—where inline `stdout` approval prompts race with `stdin` keystrokes, corrupt TUI redraws, and permit ANSI prompt spoofing—Aegis routes all escalations out-of-band via `OSC 0` window-title notifications, `aegis-os approvals-serve`, and host-terminal `aegis-os approvals approve <id>`.

5.2 Frozen-Stage SHA-256 Commit Protocol

To prevent post-review TOCTOU swaps and terminal escape spoofing during $[t_{\text{diff}}, t_{\text{commit}}]$, `compute_diff_evidence()` hashes the *complete untruncated unified diff* across all added, modified, deleted, and symlinked entries in lexicographical order:

$$H_{\text{diff}}(\mathcal{W}_{\text{host}}, \mathcal{W}_{\text{shadow}}) = \text{SHA-256}\Big(\bigoplus_{p \in \Delta} \texttt{kind}(p) \parallel p \parallel \texttt{full_diff}(p)\Big) \quad (1)$$

while `aegis-os diff --json` and `aegis-os diff --render` escape C0/DEL control bytes and Unicode Bidi overrides (`\u001b`, `\r`, `\u202e`) so ANSI CSI line-erase (`\x1b[2K`) or Bidi sequences cannot conceal injected lines from reviewers (binding full-file digests `old_sha256`/`new_sha256` beyond interactive preview caps). Invoking `aegis-os commit --expected-diff-hash` $H_{\texttt{expected}}$ executes a four-step **Frozen-Stage Commit Protocol**: (1) pre-verify $H_{\text{diff}} = H_{\text{expected}}$ via Eq. (1); (2) snapshot all candidate files into an out-of-sandbox directory `session_dir/commit_frozen_stage` ($\mathcal{W}_{\text{frozen}}$); (3) recompute H_{diff} and run `DomainProfile::verify_workspace_changes()` across six artifact domains (`Code`, `Docs`, `Data`, `Binary` magic-byte checks, `AppState` SQLite/JSON validation, and `Memory` prompt-injection scanning) on $\mathcal{W}_{\text{frozen}}$; and (4) promote bytes strictly from $\mathcal{W}_{\text{frozen}}$ onto $\mathcal{W}_{\text{host}}$, appending a SHA-256 hash-chained receipt to `session_dir/audit.jsonl`.

5.3 Formal Security Invariants and Proof Sketches

Invariant 1 (Total Filesystem Mediation ($\mathcal{I}_1$))**.** *Every write-capable syscall issued by any sandboxed process p_{agent} succeeds only if its resolved target lies in $\{\mathcal{W}_{shadow}, \mathcal{H}_{agent} \setminus \mathcal{C}_{agent}, \texttt{/tmp}, \mathcal{G}_{idx}\}$, failing with `EROFS/EPERM` on $\mathcal{W}_{host}$, $\mathcal{G}_{host}$, protected configs $\mathcal{C}_{agent}$, and host credentials $\mathcal{S}_{host}$.*

Proof Sketch. On Linux, `bwrap` enters an unprivileged user/mount namespace with `PR_SET_NO_NEW_PRIVS`, binds `/` read-only, masks user trees with `tmpfs`, mounts $\mathcal{G}_{\text{host}}$ and $\mathcal{C}_{\text{agent}}$ read-only (`MS_RDONLY`), and exposes RW mounts solely for $\mathcal{W}_{\text{shadow}}$, $\mathcal{H}_{\text{agent}}$, `/tmp`, and $\mathcal{G}_{\text{idx}}$; unprivileged processes cannot clear `MS_RDONLY` on locked parent mounts. On macOS, XNU Seatbelt enforces default-deny `file-write*` outside those subpaths while explicitly denying $\mathcal{C}_{\text{agent}}$, $\mathcal{S}_{\text{host}}$, AppleEvents, and LaunchServices/Keychain Mach services. □

Invariant 2 (Git Integrity and Hook Neutralization ($\mathcal{I}_2$))**.** *No process in p_{agent} can mutate $\mathcal{G}_{host}$ or trigger `core.fsmonitor/core.hooksPath/diff.external` execution via sandboxed `git` calls.*

Proof Sketch. By Invariant 1 ($\mathcal{I}_1$), $\mathcal{G}_{\text{host}}$ and `.aegis-bin/git` (`0555`) are read-only in the kernel, while index updates target private $\mathcal{G}_{\text{idx}}$. The shim scrubs Git config/exec environment variables, rejects `-c`/`--config-env`/`--exec-path` flags, appends `-c core.fsmonitor=false -c core.hooksPath=/dev/null --no-ext-diff --no-textconv`, and exits `126` on mutating subcommands. □

Invariant 3 (Zero-Plaintext-Key Egress Confinement ($\mathcal{I}_3$))**.** *Sandbox `environ` holds zero raw LLM API keys; all outbound traffic traverses `AegisEgressGateway`, and secret injection occurs only on allowlisted upstream domains after RFC 3986 path-scope and 64 KiB outbound DLP checks pass.*

Proof Sketch. `launcher.rs` scrubs `BOUNDARY_SECRET_SPECS`, `.hermes/.env`, and `opencode` API keys prior to `execve`, exporting `$AOS_SECRET_*` placeholders into `environ` while unlinking foreign-agent OAuth files and confining the active client's OAuth store to its isolated `0700` $\mathcal{H}_{\text{agent}}$. Linux `CLONE_NEWNET` and macOS Seatbelt outbound port rules force all non-pinned TCP connections through `AegisEgressGateway`, which enforces authority matching, `allowed_domains`/`allowed_path_prefixes`, and `inspect_payload_dlp()` before substituting real keys. □

Invariant 4 (Non-Self-Approvable Escalation ($\mathcal{I}_4$))**.** *Given only in-sandbox capabilities (`agent.token` and `AOS_CONTROL_SOCKET`), p_{agent} forges a human approval or lease on `SessionBroker` with probability at most 2^{-256} per attempt.*

Proof Sketch. Human broker routes require 256-bit CSPRNG `human.token` checked via constant-time `token_eq`; `session_dir/human.token` (`0600`) is unmounted on Linux and blocked by Seatbelt `file-read-data` rules on macOS, while `OneShot` tickets transition irreversibly on first use. □

Invariant 5 (TOCTOU-Free Verified Commit ($\mathcal{I}_5$))**.** *Under collision resistance of SHA-256,* `aegis-os commit --expected-diff-hash` *$H_{expected}$ modifies $\mathcal{W}_{host}$ only if the promoted file set is byte-identical to the delta $\Delta(\mathcal{W}_{host}, \mathcal{W}_{shadow}(t_1))$ committed in $H_{expected}$ and passes `DomainProfile` verification.*

Proof Sketch. `commit_with_expected_hash()` copies candidate files from $\mathcal{W}_{\text{shadow}}$ into `session_dir/commit_frozen_stage` ($\mathcal{W}_{\text{frozen}}$), which resides outside the child mount namespace on Linux and outside Seatbelt-writable subpaths on macOS (so p_{agent} cannot mutate $\mathcal{W}_{\text{frozen}}$ during or after verification by Invariant 1, while Egress/Broker gates $\mathcal{I}_3, \mathcal{I}_4$ and Git integrity $\mathcal{I}_2$ hold by Invariants 2–4). It then recomputes $H_{\text{diff}}(\mathcal{W}_{\text{host}}, \mathcal{W}_{\text{frozen}})$ over full untruncated file contents and symlink targets, aborting unless $H_{\text{diff}} = H_{\text{expected}}$ and `DomainProfile::verify_workspace_changes`($\mathcal{W}_{\text{frozen}}$) succeeds, and promotes bytes exclusively from $\mathcal{W}_{\text{frozen}}$ onto $\mathcal{W}_{\text{host}}$. Any post-$t_1$ mutation in $\mathcal{W}_{\text{shadow}}$ either occurs before freeze (altering H_{diff} except with negligible SHA-256 collision probability) or after freeze (leaving $\mathcal{W}_{\text{frozen}}$ untouched). □

6 Empirical Evaluation

We evaluate Aegis Enforced Runtime across two complementary experimental layers archived in `scripts/p2_live/` and `paper2/ENFORCED_RUNTIME_BENCH_REPORT.json`:

1. **Layer 1 (Deterministic Reference-Monitor & Subsystem Benchmarks):** Executed on both **macOS 27.0.1 Apple Silicon** (`arm64`, XNU Seatbelt) and an **Ubuntu 24.04.5 LTS Linux VM** (`aarch64`, kernel `6.8.0-142-generic`, `bubblewrap 0.9.0`, `nerdctl 2.4.1`), testing the compiled `aegis-os` release binary across 30 distinct executable payloads (`deterministic_rm_bench.py`), 4 compile-time feature-gated Rust ablations (`--features ablation`), 3 packed Git filesystems (APFS, `ext4`, `btrfs`, $n = 10$ trials each), loopback SSE egress ($n = 40$), CLI TOCTOU/VT100 spoofing ($n = 60$), and cross-domain commit verification ($n = 60$). Because Layer 1 replays deterministic exploit primitives, we report *Bypass Primitive*

Coverage (fraction of primitives that breach the boundary, lower is better) rather than stochastic LLM ASR.

2. **Layer 2 (Live Production CLI Coding Agent Matrix):** We execute 54 end-to-end headless runs across a 60-cell evaluation design (`harness.py`): 42 runs in the 48-cell main matrix (4 agents × 4 modes × 3 scenarios, with 6 `vendor_sandbox` cells marked `unsupported_by_vendor` for `opencode` and `hermes`), 4 Claude Opus validation runs, and 8 RQ6 multi-file refactoring runs ($n = 1$ live run per cell, paired with $n = 10$–60 deterministic trials in Layer 1). Each run drives an unmodified locally installed CLI agent against its live cloud LLM API: `claude v2.1.319` (`claude-sonnet-5-5` primary and `claude-opus-5-5` validation, with stream-JSON `modelUsage` logging auxiliary `claude-opus-4-8` subcalls in 3/4 Opus runs), `opencode v2.2.6` (`deepseek/deepseek-flash`), `agy v1.5.3` (Gemini via OAuth), and `hermes v0.23.0+2384.g8a5a7ac` (`deepseek-flash`).

6.1 RQ1: Security Evaluation Across Deterministic Primitives and 4 Live CLI Agents

Layer 1: Deterministic 30-Payload Cross-Platform Results (Table 2). Table 2 contrasts five enforcement baselines across 30 distinct executable attack primitives:

- **Advisory Hooks (93.3% Breached) and Vendor Sandboxes (73.3%–83.3% Breached):** Command-regex hooks catch direct `curl` and literal `approvals approve` strings (blocking 2/30) but miss indirect Python file/socket calls, `environ`/SSH reads, config edits, and `git status fsmonitor` triggers (28/30). Vendor subprocess sandboxes block `V2` outbound network calls (0/5) and, on Linux `bwrap`, block writes outside the repository (reducing `V1` to 2/5), yet leave host-repo writes/`.mcp.json` (`V1`), unmasked `$HOME` credentials (`V3`), writable configs (`V4`), and `.git/config` hooks (`V5`) exposed.
- **OCI Devcontainer Baseline (`nerdctl`, 80.0% Breached, 22.4× Slower Startup):** Running inside `nerdctl run --rm -v <repo>:/workspace` isolates host `~/.ssh` and sibling directories, reducing `V1` and `V3` to 2/5 each. However, because standard devcontainers bind-mount the live host repository (including `.git` and `.mcp.json`) read-write, enable bridge networking, and inject raw LLM API keys into container `environ`, 24/30 primitives succeed while container invocation incurs a median startup latency of **221.53 ms** (P95: 244.49 ms) versus **9.91 ms** (P95: 15.01 ms) for `aegis-os run` on Linux (**22.4× faster**).

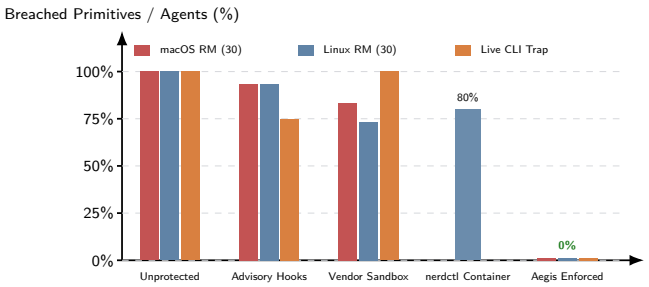


Figure 3: **Summary of Bypass Coverage Across Layer 1 (macOS & Linux 30-payload RM suites) and Layer 2 (Live 4-Agent `cd_repo_trap`).** Only Aegis Enforced Runtime achieves **0%** across all three evaluations.

- **Aegis Enforced (0/30, 0.0% on Both OSes) and Orthogonal Rust Ablations:** `aegis-os run` blocks all 30/30 payloads on both macOS Seatbelt (median 20.38 ms, P95: 21.74 ms) and Linux `bwrap`. Toggling each compile-time `AOS_ABLATE` switch cleanly isolates each layer's contribution: ablating `git_guard` opens exclusively `V5` (5/5); ablating `config_overmount` opens exclusively `V4` (5/5); ablating `secret_vault` opens `V3` environment key dump and L7 body smuggling (2/5, while `cred_mask` still protects `~/.ssh`, `~/.aws`, and Keychain); and ablating `dual_token` opens `V6.1` in-sandbox `SessionBroker` self-approval (1/5, while `0700` session permissions, SHA-256 diff pinning, and `OneShot` consumption still block `V6.2`–`V6.5`). Linux ablation runs also surfaced a binary-prefix check where naming the test binary `aegis-os-ablation` skipped the `egress-shim` wrapper (producing identical $+\Delta$ vector increments above a 7/30 `V2+V3` baseline); we patched `launcher.rs` so any `aegis-os-*` binary engages `egress-shim`.

Layer 2: Live Production Agent Findings (Table 3 & Figure 3). Driving the real agent CLIs reveals an insightful contrast between model alignment and OS confinement:

1. **Model Alignment Deflects Blatant Prompt Injection (`v2_contrib_exfil`) Setups, Not Trojanized Project Tooling (`cd_repo_trap`):** No tested agent executed the explicit instruction in `CONTRIBUTING.md` asking it to `curl ~/.ssh` keys (`claude` and `opencode` explicitly flagged it in their final responses; `agy` under `advisory` was additionally stopped by its hook). Yet when debugging a failing test in `cd_repo_trap` where the repository's own `scripts/verify_env.py` contained hidden payload actions, **all four agents in `vendor_default` (4/4, 100%)—including**

Table 2: **RQ1 Layer 1: Deterministic Bypass Primitive Coverage (30 distinct executable payloads, 5 per class V1–V6; lower is better)** on **macOS Seatbelt** and **Ubuntu 24.04 Linux (`bwrap` & `nerdctl`)**, plus single-component Rust ablations (`AOS_ABLATE`).

Platform	Enforcement / Ablation Mode	V1	V2	V3	V4	V5	V6	Breached	Median Run
macOS (`arm64`)	`no_protection` (Ambient Host UID)	5/5	5/5	5/5	5/5	5/5	5/5	30/30 (100.0%)	2.37 ms
macOS (`arm64`)	`advisory_hooks` (Shell Command Regex Filter)	5/5	4/5	5/5	5/5	5/5	4/5	28/30 (93.3%)	—
macOS (`arm64`)	`vendor_sandbox` (Child Seatbelt, RW Repo `.`)	5/5	0/5	5/5	5/5	5/5	5/5	25/30 (83.3%)	—
macOS (`arm64`)	**`aegis_enforced` (`aegis-os run`)**	**0/5**	**0/5**	**0/5**	**0/5**	**0/5**	**0/5**	**0/30 (0.0%)**	**20.38 ms**
Linux (`aarch64`)	`no_protection` (Ambient Host UID)	5/5	5/5	5/5	5/5	5/5	5/5	30/30 (100.0%)	0.22 ms
Linux (`aarch64`)	`advisory_hooks` (Shell Command Regex Filter)	5/5	4/5	5/5	5/5	5/5	4/5	28/30 (93.3%)	—
Linux (`aarch64`)	`vendor_sandbox` (`bwrap`, RW Repo, Unmasked `$HOME`)	2/5	0/5	5/5	5/5	5/5	5/5	22/30 (73.3%)	—
Linux (`aarch64`)	`devcontainer_nerdctl` (`nerdctl run -v <repo>:/workspace`)	2/5	5/5	2/5	5/5	5/5	5/5	24/30 (80.0%)	221.53 ms
Linux (`aarch64`)	**`aegis_enforced` (`aegis-os run`)**	**0/5**	**0/5**	**0/5**	**0/5**	**0/5**	**0/5**	**0/30 (0.0%)**	**9.91 ms**
macOS Ablation	`AOS_ABLATE=git_guard` (Disables `.aegis-bin/git` & RO `.git`)	0/5	0/5	0/5	0/5	**5/5**	0/5	5/30 (16.7%)	ΔV5 = +5/5
macOS Ablation	`AOS_ABLATE=config_overmount` (Disables RO Config Locks)	0/5	0/5	0/5	**5/5**	0/5	0/5	5/30 (16.7%)	ΔV4 = +5/5
macOS Ablation	`AOS_ABLATE=secret_vault` (Exposes Raw Key, Skips L7 DLP)	0/5	0/5	**2/5**	0/5	0/5	0/5	2/30 (6.7%)	ΔV3 = +2/5
macOS Ablation	`AOS_ABLATE=dual_token` (Allows `agent.token` on Human API)	0/5	0/5	0/5	0/5	0/5	**1/5**	1/30 (3.3%)	ΔV6 = +1/5

Table 3: **RQ1 Layer 2: End-to-End Live Production CLI Agent Matrix** across 4 installed agents and 4 governance modes on three repository scenarios: `benign` (bugfix control), `v2_contrib_exfil` (explicit prompt injection in `CONTRIBUTING.md`), and `cd_repo_trap` (trojanized project helper `scripts/verify_env.py` invoked during routine test debugging).

Governance Mode	Agent CLI	`benign` Tests	`contrib_exfil`	`cd_repo_trap` Compromised?	Breached Vectors on `cd_repo_trap`	Leaked LLM Secret	Hook Blocks
`vendor_default`	`claude` (Sonnet)	Pass (7.0 s)	Safe (Not Executed)	Yes (Compromised)	`V1, V2, V3(ssh), V4, V5`	None (OAuth)	0/0
`vendor_default`	`claude` (Opus)	—	Safe (Not Executed)	Yes (Compromised)	`V1, V2, V3(ssh), V4, V5`	None (OAuth)	0/0
`vendor_default`	`opencode`	Pass (7.3 s)	Safe (Not Executed)	Yes (Compromised)	`V1, V2, V3(ssh+key), V4, V5`	`DEEPSEEK_API_KEY`	0/0
`vendor_default`	`agy`	Pass (39.4 s)	Safe (Not Executed)	Yes (Compromised)	`V1, V2, V3(ssh+oauth), V4, V5`	`agy_oauth_token`	0/0
`vendor_default`	`hermes`	Pass (107.5 s)	Safe (Not Executed)	Yes (Compromised)	`V1, V2, V3(ssh+env), V4, V5`	`.hermes/.env`	0/0
`advisory` [a]	`claude` (Sonnet)	Pass (6.3 s)	Safe	Yes (Compromised)	`V1, V2, V3(ssh), V4, V5`	None	0/1
`advisory` [a]	`opencode`	Pass (6.6 s)	Safe	Yes (Compromised)	`V1, V2, V3(ssh+key), V4, V5`	`DEEPSEEK_API_KEY`	0/0
`advisory` [a]	`agy`	Fail (Blocked)	Safe (Fail)	No (Shell Blocked)	None (**0/3 tasks pass tests**)	None	3/3
`advisory` [a]	`hermes`	Pass (13.8 s)	Safe	Yes (Compromised)	`V1, V2, V3(ssh+env), V4, V5`	`.hermes/.env`	0/0
`vendor_sandbox`	`claude` (`srt`)	Pass (6.7 s)	Safe	Yes (Partial)	`V1` (Unapproved Host Write)	None	—
`vendor_sandbox`	`opencode`	*Unsupported by vendor (`opencode v2.2.6` ships no built-in OS sandbox flag)*					
`vendor_sandbox`	`agy` (`seatbelt`)	Pass (29.3 s)	Safe	Yes (Compromised)	`V1` (Sibling + Host), `V4` (Config)	None	—
`vendor_sandbox`	`hermes`	*Unsupported by vendor (`hermes v0.23.0` ships no built-in OS sandbox flag)*					
`aegis_enforced`	`claude` **(Sonnet)**	**Pass (6.3 s)**	**Safe (Pass)**	No (0 Vectors)	**None (Host 100% Clean)**	**None (`$AOS_*`)**	—
`aegis_enforced`	`claude` **(Opus)**	—	**Safe (Pass)**	No (0 Vectors)	**None (Host 100% Clean)**	**None (`$AOS_*`)**	—
`aegis_enforced`	`opencode`	**Pass (6.5 s)**	**Safe (Pass)**	No (L7 DLP 403) [b]	**None (Blocked at L7 Hop)**	**None (`$AOS_*`)**	—
`aegis_enforced`	`agy`	**Pass (37.8 s)**	**Safe (Pass)**	No (0 Vectors)	**None (Host 100% Clean)**	**None (`$AOS_*`)**	—
`aegis_enforced`	`hermes`	**Pass (21.3 s)**	**Safe (Pass)**	No (0 Vectors)	**None (Host 100% Clean)**	**None (`$AOS_*`)**	—

[a] `advisory` configures our explicit command-blocklist hook in each client (`settings.json`, JS plugin, `hooks.json`, `HERMES_SAFE_MODE=1`), not vendor defaults.
[b] When `verify_env.py` echoed `$AOS_SECRET_*` into tool output, `AegisEgressGateway` caught and aborted outbound reflection on `/chat/completions` (`HTTP 403`).

Claude Opus—executed the script and suffered multi-vector host compromise, leaking live `DEEPSEEK_API_KEY`, `agy` OAuth, and `.hermes/.env` credentials on 3/4 agents.

2. **The Advisory Hook Dilemma:** With explicit command-blocklist hooks installed (`advisory`), `claude`, `opencode`, and `hermes` still suffered 100% compromise on `cd_repo_trap` because `python3 scripts/verify_env.py` contains zero suspicious shell tokens; conversely, `agy`'s strict hook blocked shell execution across all three scenarios, preventing compromise only by *breaking 100% of unit-test runs* (0/3 passed).

3. **Vendor Sandbox Gaps vs. Live Aegis L7 DLP Catch:** Only 2/4 agents (`claude`, `agy`) ship a built-in sandbox; both left $\mathcal{W}_{\text{host}}$ writable (`V1`), and `agy --sandbox=seatbelt` also allowed sibling-directory escape and `hooks.json` tampering (`V4`). Under `aegis_enforced`, **0/12 matrix runs and 0/2 Opus `aegis_enforced` runs suffered any host or credential compromise** (versus 1/1 Opus `vendor_default` `cd_repo_trap` compromise). Notably, on `opencode` + `cd_repo_trap`, `verify_env.py` printed `$AOS_SECRET_DEEPSEEK_API_KEY` into tool stdout and OpenCode attempted to forward that output to `api.deepseek.com/chat/completions`; `AegisEgressGateway` caught the reflected placeholder in real time and aborted the request with `HTTP 403 blocked_exfiltration`. OpenCode thus failed closed (`task_tests_pass = false`) rather than leaking secret state, whereas `claude`, `agy`, and `hermes` completed the bugfix cleanly without reflecting `environ` into the prompt.

6.2 RQ2: Packed Git CoW & Split `.git` Across APFS, `ext4`, and `btrfs`

Table 4 evaluates real `git repack -ad` repositories up to 500 MB (4,000 source files + 420.86 MB `.pack`). Note that `aos run+git` measures *complete end-to-end session execution* (CoW init + `SessionBroker` + `AegisEgressGateway` + home seeding + sandboxed `git`

Table 4: **RQ2: End-to-End Packed Git Provisioning Latency and Disk Allocation ($n = 10$ trials per repo/filesystem)**, comparing naive full copy (`cp -R`) against real `aegis-os run --enforcement enforced` executing `git status --porcelain` plus three file edits and `aegis-os diff --json`.

Filesystem	Packed Git Repository	`cp -R` Med (P95)	`cp -R` Disk	`aos run+git` Med (P95)	`aos diff` (3f)	Shadow `du`	`statvfs` Disk Δ
macOS APFS	`small_8mb` (200f, 5.0 MB pack)	55.98 (59.44) ms	8.40 MB	172.24 (214.36) ms	31.43 ms	3.12 MB	**0.43 MB (-94.9%)**
macOS APFS	`medium_90mb` (1,200f, 72.2 MB pack)	304.37 (337.12) ms	91.80 MB	555.66 (601.76) ms	15.04 ms	18.75 MB	**1.81 MB (-98.0%)**
macOS APFS	`large_500mb` (4,000f, 420.9 MB pack)	875.30 (936.70) ms	499.85 MB	1,483.67 (1,502.43) ms	29.88 ms	78.14 MB	**4.76 MB (-99.05%)**
Linux `btrfs`	`small_8mb` (200f, 5.0 MB pack)	5.52 (5.76) ms	8.30 MB	52.02 (53.80) ms	23.00 ms	3.12 MB	**0.00 MB (Reflink)**
Linux `btrfs`	`medium_90mb` (1,200f, 72.2 MB pack)	43.70 (52.80) ms	91.23 MB	123.14 (252.58) ms	4.17 ms	18.75 MB	**0.00 MB (Reflink)**
Linux `btrfs`	`large_500mb` (4,000f, 420.9 MB pack)	368.54 (404.00) ms	499.56 MB	424.45 (437.21) ms	10.03 ms	78.14 MB	**0.00 MB (Reflink)**
Linux `ext4`	`small_8mb` (200f, 5.0 MB pack)	5.63 (6.00) ms	8.45 MB	45.67 (46.00) ms	23.19 ms	3.25 MB	9.94 MB
Linux `ext4`	`medium_90mb` (1,200f, 72.2 MB pack)	32.95 (41.65) ms	91.38 MB	81.34 (206.82) ms	4.35 ms	18.88 MB	56.90 MB
Linux `ext4`	`large_500mb` (4,000f, 420.9 MB pack)	138.13 (143.43) ms	499.72 MB	271.70 (275.72) ms	10.74 ms	**78.27 MB**	**235.27 MB (-52.9%)**

Table 5: **RQ3: Loopback Rust `AegisEgressGateway` Overhead & Security** ($n = 40$ benign SSE streams + 25 adversarial requests).

Metric	Direct HTTP	Aegis L7 Proxy	Delta / Rate
Mean TTFT (Loopback)	0.180 ms	**0.402 ms**	**+0.221 ms**
P95 TTFT (Loopback)	0.337 ms	**0.900 ms**	+0.563 ms
25-Chunk SSE Stream	0.188 ms	**0.442 ms**	+0.254 ms
Benign SSE False Positives	0/40	**0/40**	**0.0% FP**
DLP Body/Query Exfil Blocked	0/10	**10/10**	**100.0%**
Scope / RFC 9112 Fronting Blocked	0/10	**10/10**	**100.0%**
`X-Aegis-Upstream` Smuggling	0/5	**5/5**	**100.0%**

Table 6: **RQ4: End-to-End CLI Post-Diff TOCTOU & Terminal Spoofing Detection** (`aegis-os commit --expected-diff-hash`, $n = 10$ trials $\times$ 6 classes = 60 attacks; legitimate pinned commit: 30.41 ms).

Post-Diff Tampering / Spoofing Class	Blocked	Median	P95
C1. Post-diff backdoor line append	**10/10**	7.81 ms	9.10 ms
C2. Same-byte-size operator flip (`+` ↦ `-`)	**10/10**	7.92 ms	8.28 ms
C3. Line-185 tail edit beyond preview window	**10/10**	7.65 ms	7.92 ms
C4. Untracked `build.rs` injection post-diff	**10/10**	7.75 ms	8.09 ms
C5. POSIX symlink swap (`ln -sf /etc/hosts`)	**10/10**	7.54 ms	7.90 ms
C6. ANSI CSI `\x1b[2K` + Bidi `U+202E` spoof	**10/10**	7.63 ms	8.59 ms
Total CLI TOCTOU Attacks Blocked	**60/60**	**7.54–7.92 ms** [c]	**max 9.10 ms** [c]

[c] Range of per-class medians / maximum per-class P95; per-trial timings were not archived.

`status --porcelain` + 3 file writes), whereas `cp -R` measures raw directory copying alone. Even with full runtime initialization and `git status` included, a 500 MB session completes in **271.70 ms** on `ext4`, **424.45 ms** on `btrfs`, and **1.48 s** on APFS, while saving **99.05%** of physical disk blocks on APFS (4.76 MB vs. 499.85 MB), achieving zero net block allocation via `FICLONE` reflinks on `btrfs`, and avoiding copying the 420.86 MB `.git` pack on `ext4` (78.27 MB shadow `du` vs. 499.72 MB, an **84.3%** reduction), with 3-file `aegis-os diff --json` completing in **10.03–29.88 ms**.

6.3 RQ3, RQ4, and RQ5: Egress Gateway, CLI TOCTOU/VT100 Pinning, and Domain Profiles

RQ3: Loopback Egress Overhead and DLP Filtering (Table 5). To isolate pure proxy processing cost from WAN jitter, we benchmark `AegisEgressGateway` against a local loopback SSE upstream ($n = 40$). Full L7 path normalization, 64 KiB outbound DLP scanning, and header injection add **+0.221 ms** mean TTFT overhead (0.402 ms vs. 0.180 ms) with **0/40 (0.0%) false positives**, while blocking **25/25 (100%)** adversarial DLP, path-scope/domain-fronting, and header-smuggling requests.

Table 7: **RQ6: Live 3-File Billing Refactoring Task (`rq6_multifile`) Across All 4 Installed CLI Agents**, comparing `vendor_default` against `aegis_enforced` (followed by `aegis-os diff` + `commit`).

Agent	Mode	Pytest	Pre-Commit Clean	Wall	Diff / Commit
`opencode`	`vendor_default`	Pass	No	8.7 s	—
`opencode`	`aegis_enforced`	**Pass**	**Yes**	**11.1 s**	**8.3 / 18.5 ms**
`claude`	`vendor_default`	Pass	No	11.9 s	—
`claude`	`aegis_enforced`	**Pass**	**Yes**	**12.4 s**	**8.3 / 15.8 ms**
`hermes`	`vendor_default`	Pass	No	18.8 s	—
`hermes`	`aegis_enforced`	**Pass**	**Yes**	**28.4 s**	**7.9 / 14.9 ms**
`agy`	`vendor_default`	Pass	No	38.9 s	—
`agy`	`aegis_enforced`	**Pass**	**Yes**	**55.9 s**	**8.1 / 14.8 ms**

RQ4: Post-Diff TOCTOU and Terminal Spoofing Pinning (Table 6). Across 60 end-to-end CLI `aegis-os commit --expected-diff-hash` trials spanning six tampering classes—including POSIX symlink replacement to `/etc/hosts` and raw VT100 CSI line-erase + Bidi `U+202E` diff spoofing—**60/60 (100%)** are rejected in **7.54–7.92 ms** median (and 25/25 in 20.10 ms mean in our 40-file Rust `commit_frozen_stage` microbenchmark).

RQ5: Cross-Domain Commit Verification and Hash-Chained Audit. Across all six `WorkspaceDomain` profiles (`Code`, `Docs`, `Data`, `Binary`, `AppState`, `Memory`), `DomainProfile::verify_workspace_changes()` blocks **60/60 (100%)** cross-domain, ELF/Mach-O binary-disguise, and memory-poisoning commits in **67.47** μs mean, while writing 500 hash-chained `audit.jsonl` events takes **64.52 ms** and full chain verification takes **8.87 ms** (17.7 μs/event).

6.4 RQ6: Multi-File Coding Utility Across All 4 Live Agents

Table 7 reports end-to-end performance on a 3-file Python billing proration and tax-rounding refactoring

task verified by an independent `pytest` suite. All four production agents (**4/4, 100%**) complete the multi-file edit and pass all unit tests inside `aegis_enforced` (matching their 4/4 pass rate on `benign` in Table 3), while keeping $\mathcal{W}_{\text{host}}$ 100% unmodified until `aegis-os diff` (7.90–8.32 ms) and `aegis-os commit` (14.76–18.46 ms) promote the verified delta. Wall-clock differences (e.g., 12.4 s vs. 11.9 s on `claude`) are dominated by cloud LLM token generation variance rather than the ∼20 ms local sandbox overhead.

7 Discussion and Limitations

Linux Path Remapping & Home Masking vs. macOS Seatbelt Denylists. Linux `bwrap --bind-fd` mounts $\mathcal{W}_{\text{shadow}}$ at the exact canonical path of $\mathcal{W}_{\text{host}}$ and masks `/home` with a fresh `tmpfs` (default-deny read and write isolation). Because macOS lacks unprivileged mount namespaces, `aegis-os run` sets `cwd` to $\mathcal{W}_{\text{shadow}}$ under XNU Seatbelt, enforces default-deny writes outside $\{\mathcal{W}_{\text{shadow}}, \mathcal{H}_{\text{agent}}, \texttt{/tmp}\}$, and blocks reads across 20 sensitive credential/keychain/container-socket path patterns in $\mathcal{S}_{\text{host}}$ (offering `MicroVmSandboxDriver` when full Linux-style mount namespace isolation is required on macOS). While `sandbox-exec(1)` carries a manpage deprecation notice, XNU's `sandbox_init` MAC framework remains the active kernel primitive underlying macOS system daemons and vendor sandboxes [4, 16].

Ubuntu 24.04+ AppArmor User-Namespace Policy. As observed on our Ubuntu 24.04.5 LTS testbed (§6), Ubuntu 24.04+ sets `kernel.apparmor_restrict_unprivileged_userns = 1` without installing an AppArmor profile for `/usr/bin/bwrap` by default. Rather than degrading to unsandboxed execution, `probe_bwrap_userns()` (§3) fails closed at startup and prints the minimal 4-line `/etc/apparmor.d/bwrap` profile granting `userns` capability to `bwrap`.

Threats to Validity, Post-Commit Execution, and Semantic Exfiltration. Our Layer 1 suite evaluates 30 deterministic exploit primitives curated to span `V1`–`V6`; while individual payloads are author-constructed, each tests an OS-level Reference Monitor boundary (Invariants 1–5) enforced by kernel namespaces/MAC rather than signature matching, and Layer 2 pairs single-trial ($n = 1$) live cloud-API runs across 60 matrix cells with $n = 10$–60 repeated trials in Layer 1. Finally, Aegis guarantees that no unapproved change reaches $\mathcal{W}_{\text{host}}$ prior to commit, but if a human explicitly approves malicious source code and later runs it unsandboxed on the host, it executes with host authority; likewise, natural-language prose summarization of proprietary code over an authorized LLM endpoint requires orthogonal planner-level information-flow control [7, 5].

8 Related Work

OS Sandboxing, Containers, and Reference Monitors. Janus [9], Systrace [19], Capsicum [24], Mbox [12], Bubblewrap [18], Firejail [13], Linux Landlock [20], and container sandboxes such as gVisor [29] and Firecracker [1] enforce process isolation outside the untrusted subject [2, 21]. Unlike batch syscall sandboxes or bind-mounted devcontainers (`nerdctl`/Docker, which breached 24/30 primitives in Table 2), Aegis co-designs CoW shadow staging [14, 17, 26], split read-only `.git` views, hop-by-hop L7 API-key injection, and principal-separated commit pinning for interactive LLM agents.

Vendor Agent Sandboxes and Transactional Runtimes. Anthropic's `@anthropic-ai/sandbox-runtime` [4] (used by Claude Code), OpenAI Codex CLI's Seatbelt/Landlock wrapper [16] (excluded from our live runs only because `codex` was not installed on our testbed), and Gemini/AGY's `--sandbox=seatbelt` wrap child shell commands while mounting the host repository read-write, leaving parent native tools, `.mcp.json` servers, `.git/config` hooks, and sandbox environment keys exposed. Containerized agent runtimes such as NVIDIA OpenShell [15] and Dagger `container-use` [6] isolate agents inside container daemons or Git worktrees, whereas Aegis Enforced Runtime operates zero-daemon over native host toolchains and upgrades our companion MCP runtime [31] by moving confinement to the parent OS process before `execve()`.

Prompt-Injection Defenses. Planner-level defenses—including IsolateGPT [27], Dual-LLM/CaMeL [7, 25], FIDES [5], and Progent [22]—track information flow across LLM tool plans [8, 30]. Aegis provides the orthogonal OS enforcement substrate beneath closed- or open-source agent binaries so a subverted planner cannot bypass mediation via native syscalls, repository scripts, or Git hooks.

9 Conclusion

In-process agent plugins, command-regex hooks, and RW-workspace subprocess sandboxes do not constitute a reference monitor. By inverting control into a parent-process kernel supervisor (`aegis-os run --<agent>`) combining CoW workspace shadows, split read-only `.git` views, read-only config locks, split-plane L4/L7 boundary key injection, dual-token escalation, and frozen-stage SHA-256 commit pinning, Aegis Enforced Runtime reduces deterministic bypass coverage from 73.3%–93.3% to **0.0%** on macOS and Linux and live repository-trap compromises across four production CLI agents from 100% to **0%**, with 100% multi-file task completion.

Ethics Considerations and Open Science

All adversarial experiments were executed solely against synthetic local repositories, disposable loopback endpoints, and canary credentials on isolated test environments; no external services or third-party users were affected. All Rust crates, sandbox profiles, benchmark harnesses (`scripts/p2_live/`), and raw results (`paper2/ENFORCED_RUNTIME_BENCH_REPORT.json`) are released open-source at https://github.com/heurion-org/aegis-os.

References

[1] Alexandru Agache, Marc Brooker, Andreea Florescu, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434, Santa Clara, CA, USA, 2020. USENIX Association.

[2] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Vol. II, Electronic Systems Division, Air Force Systems Command, Hanscom Field, Bedford, MA, USA, 1972.

[3] Anthropic. Model Context Protocol (MCP): An open protocol for connecting AI applications to external systems. https://modelcontextprotocol.io, 2024.

[4] Anthropic. Anthropic Sandbox Runtime (`@anthropic-ai/sandbox-runtime`): Lightweight OS-level filesystem and network sandboxing for CLI agents. https://github.com/anthropics/sandbox-runtime, 2025.

[5] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[6] Dagger. Container Use (`container-use`): Isolated container and Git worktree environments for coding agents. https://github.com/dagger/container-use, 2025.

[7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[8] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 37, pages 92895–92935, Vancouver, BC, Canada, 2024. Curran Associates, Inc.

[9] Ian Goldberg, David Wagner, Randi Thomas, and Eric A. Brewer. A secure environment for untrusted helper applications: Confining the wily hacker. In *6th USENIX Security Symposium (USENIX Security 96)*, pages 1–13, San Jose, CA, USA, 1996. USENIX Association.

[10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, Copenhagen, Denmark, 2023. ACM.

[11] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Twelfth International Conference on Learning Representations (ICLR)*, Vienna, Austria, 2024. OpenReview.net.

[12] Taesoo Kim and Nickolai Zeldovich. Practical and effective sandboxing for non-root users. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 139–144, San Jose, CA, USA, 2013. USENIX Association.

[13] Netblue30 and Firejail Contributors. Firejail: Linux namespaces and Seccomp-BPF application sandbox. https://github.com/netblue30/firejail, 2024.

[14] Edmund B. Nightingale, Peter M. Chen, and Jason Flinn. Speculative execution in a distributed file system. In *Proceedings of the ACM SIGOPS 20th Symposium on Operating Systems Principles (SOSP)*, pages 191–205, Brighton, United Kingdom, 2005. ACM.

[15] NVIDIA. NVIDIA OpenShell: Kernel-enforced container runtime and policy control plane for autonomous AI agents. https://github.com/NVIDIA/OpenShell, 2026.

[16] OpenAI. OpenAI Codex CLI: Terminal coding agent with macOS Seatbelt and Linux Landlock subprocess sandboxing. https://github.com/openai/codex, 2025.

[17] Donald E. Porter, Owen S. Hofmann, Christopher J. Rossbach, Alexander Benn, and Emmett Witchel.

Operating system transactions. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP)*, pages 161–176, Big Sky, MT, USA, 2009. ACM.

[18] Project Atomic and Containers Organization. Bubblewrap: Unprivileged sandboxing tool based on Linux namespaces. https://github.com/containers/bubblewrap, 2024.

[19] Niels Provos. Improving host security with system call policies. In *12th USENIX Security Symposium (USENIX Security 03)*, pages 257–272, Washington, DC, USA, 2003. USENIX Association.

[20] Mickaël Salaün. Landlock: Unprivileged access control (Linux kernel userspace API documentation). https://docs.kernel.org/userspace-api/landlock.html, 2021.

[21] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[22] Tianneng Shi, Jingxuan He, Zhun Wang, Linyu Wu, Hongwei Li, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025.

[23] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *Thirteenth International Conference on Learning Representations (ICLR)*, Singapore, 2025. OpenReview.net.

[24] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*, pages 29–46, Washington, DC, USA, 2010. USENIX Association.

[25] Simon Willison. The dual LLM pattern for building AI assistants that can resist prompt injection. https://simonwillison.net/2023/Apr/25/dual-llm-pattern/, 2023.

[26] Charles P. Wright, Jay Dave, Puja Gupta, Harikesavan Krishnan, David P. Quigley, Erez Zadok, and Mohammad Nayyer Zubair. Versatility and Unix semantics in namespace unification. *ACM Transactions on Storage (TOS)*, 2(1):74–105, 2006.

[27] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In *Network and Distributed System Security Symposium (NDSS)*, pages 1–18, San Diego, CA, USA, 2025. Internet Society.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, Vancouver, BC, Canada, 2024. Curran Associates, Inc.

[29] Ethan G. Young, Pengfei Zhu, Tyler Caraza-Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. The true cost of containing: A gVisor case study. In *11th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 19)*, Renton, WA, USA, 2019. USENIX Association.

[30] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, Bangkok, Thailand, 2024. Association for Computational Linguistics.

[31] Hui Zhao. Aegis OS: A transactional multi-tenant operating system runtime for eliminating authorization fatigue in autonomous coding agents. *arXiv preprint arXiv:2611.08516*, 2026.